

算力网络中高效算力资源度量方法

柴若楠 郜 帅 兰江雨 刘宁春

(北京交通大学电子信息工程学院 北京 100044)
(21111030@bjtu.edu.cn)

Efficient Computing Resource Metric Method in Computing-First Network

Chai Ruonan, Gao Shuai, Lan Jiangyu, and Liu Ningchun

(School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044)

Abstract With the continuous development of new network services and the increasing demand for computing, computing-first network (CFN) has attracted people's attention and is gradually developing. As a method to measure the computing and storage capacity of various computing platforms, the computing resource metric plays an important role in achieving user awareness and efficient scheduling of computing resources in CFN. At present, the research on computing resource metrics is in its infancy. Most of those that only consider some static or dynamic indicators are relatively simple, which cannot guarantee the utilization of computing resources and the precision of matching computing resources. In this study, we design a hybrid metric method (HMM), which combines static and dynamic indicators to measure computing resources. This method takes the basic performance of the computing nodes and the dynamic changes in their working state into account. In addition, we also consider lots of static and dynamic indicators to enhance the comprehensiveness of HMM. The experiments and a large number of data analyses show that the metric method we propose has good improvement in the utilization of computing resources and the precision of matching computing resources.

Key words computing-first network; computing resources; computing resource metric; hybrid metric method

摘 要 随着新型网络业务的不断发展和对算力需求的不断提高,算力网络技术逐渐走进人们的视野并不断发展壮大。而算力度量,作为度量各类算力平台中计算和存储能力的方法,在算力网络业务感知和算力资源高效调度中扮演着重要的角色。目前算力度量的研究尚处于起步阶段,已有的度量方法相对单一,只考虑了部分静态或动态指标,难以保证算力资源利用率和算力资源匹配准确率。设计了一种先静后动的混合式度量方法(hybrid metric method, HMM),该方法结合静态和动态指标来度量算力资源,考虑了算力节点的基础性能及其动态工作状态的变化,并且在静动态的度量指标的选取上也进行了全面的考量。通过实验和数据分析证明,所提度量方法 HMM 能有效提升算力资源利用率和算力资源匹配准确率。

关键词 算力网络;算力资源;算力度量;混合式度量方法

中图法分类号 TP391

收稿日期: 2023-01-03; 修回日期: 2023-02-06

基金项目: 国家重点研发计划项目(2022YFB2901900); 国家自然科学基金项目(61972026, 61802014)

This work was supported by the National Key Research and Development Program of China (2022YFB2901900), and the National Natural Science Foundation of China (61972026, 61802014).

通信作者: 郜帅(shgao@bjtu.edu.cn)

作为一种算力和网络融合的新型网络技术和架构,算力网络(computing-first network)已成为学术界和工业界的热门话题和研究方向.算力度量(computing resource metric)是算力网络中的关键性工作,它作为算力网络底层的技术基石,是有效标识和衡量算力资源的重要方法,在算力网络中扮演着不可或缺的角色.算力网络得以发展的重要前提是对算力资源有精准的评估和管控,这需要对算力资源进行准确地标识和表征,以配合前端的用户意图感知和后端的算力资源调度决策.

然而,现在业界对于算力度量的研究尚处于起步阶段.目前的研究中对算力度量标准的设计较为单一,大部分单独使用静态指标或动态指标来衡量算力资源^[1-3],这样容易造成算力资源利用率低、匹配算力资源准确性较差等问题.

例如,文献[1]中所提到的指标均为静态指标,没有考虑实时变化的网络状态和算力节点的工作状态,导致算力资源匹配准确率较低.因为当2个算力节点的基础性能相近时,由于无法得知算力节点的实时工作状态,难以区分哪一个节点更适合用户任务,可能会分配到资源不匹配的算力节点.

与文献[1]不同,文献[2-3]中的算力度量指标均为动态指标,文献[2-3]中提到了使用计算资源大小、CPU使用率、存储情况和网络状态等动态指标来度量算力资源,但缺少对静态指标的考虑.如果不考虑静态指标,将难以区分算力节点的基础性能,可能会出现“高分低就”现象,导致算力资源利用率较低.

可以看出,单方面考虑静态指标或动态指标,可能会影响算力度量的有效性.由于静态指标代表算力节点的基础性能,动态指标代表算力节点的动态性能,结合目前业界对于算力度量研究存在的问题,为更有效地进行算力度量,提出了一种先静后动的混合式度量方法(hybrid metric method, HMM),并提出了更为全面的指标.

本文利用熵权法^[4]去分析静态指标,包括TOPS/W(tera operations per second per watt), GOPS(giga operations per second), MIPS(million instructions per second), RAM和硬盘存储大小.对每个算力节点的基础性能进行评分,然后利用基于决策树的CART算法^[5]将排好序的分数进行分段;最后在分完段的每一段内,分析CPU空闲率、GPU空闲率、剩余硬盘存储和吞吐率等指标,利用 n 维欧氏距离的方法^[6]计算算力节点与用户任务所需性能之间的“距离”,找到最短的“距离”,即找到了更可能满足用户

任务需求的算力节点.

本文的主要贡献有2点:

1)为解决目前业界对于算力度量的指标单一化、片面化等问题,提出了更为综合性的指标;

2)提出了一种结合静态指标和动态指标的混合式算力度量方法HMM.经实验和数据分析,在算力节点的选择上,HMM在算力资源利用率和匹配准确率上表现得更优.

1 相关工作

关于算力度量的设计问题,目前已经存在相关研究工作,主要从不同的算力服务场景出发,包括基础算力、智能算力和超算算力.基础算力表示以中央处理器(central processing unit, CPU)为计算单元的计算能力;智能算力是指以图形处理器(graphics processing unit, GPU)、现场可编程逻辑门阵列(field programmable gate array, FPGA)、专用集成电路(application specific integrated circuit, ASIC)等为代表的具有人工智能(artificial intelligence, AI)训练和推理学习的加速计算单元的计算能力;超算算力则是反映普遍用于科学计算场景的高性能计算集群的计算能力^[1].为更深入的研究,总结了传统资源度量技术和现有算力资源度量技术.

1.1 传统资源度量技术

文献[7]提出了一种衡量CPU性能的方法,其主流思想一直延续至今.该文提出将CPI(clock per instruction)作为衡量CPU性能的指标.由于CPI是2个可测量的东西的乘积: $CPI = \text{每条指令的周期} \times \text{每条指令的事件}$,CPI被认为是表示处理器性能的最自然的度量.文献[7]在度量CPU性能时所涉及到的指令数、时钟周期、操作数等指标,为本文提出采用TOPS/W和MIPS作为衡量CPU静态指标参数提供了启发.文献[8]提到GPU的性能与周期指令数、浮点运算能力和寄存器有关,如IPC(instructions per cycle)、缓存命中率(cache hit rate)、DRAM利用率、L2利用率等.文献[9]针对GPU集群中多项性能指标实时优化的问题,综合考虑计算速度、能耗和可靠性3项指标,利用极大熵函数法把3项指标转化为一个综合性能评价指标,并构造了控制稳定性和鲁棒性良好的模型.其中,在状态信息中只有计算频率为计算速度指标提供依据,由于本文不涉及GPU集群,因此不考虑能耗和控制问题,但文献[9]为本文采用GOPS和RAM作为静态指标参数提供了思路.文献[10]基

于泊松假设的数学模型定义了从活动模式到空闲模式的转换概率,通过不同磁盘模式之间的转换概率来研究 I/O 系统的能耗和性能指标,并评估了能耗和性能指标,包括存储器、队列长度、吞吐率、响应时间等,为本文采用 RAM、硬盘存储量和吞吐率作为静态指标参数提供了启发。

1.2 算力资源度量技术

表 1 中主要展示了目前 3 类算力度量的设计,即静态指标、动态指标和静态动态结合的混合指标。

Table 1 Comparison of Computing Resource Metric Methods

表 1 算力度量方法的对比

相关工作方法	静态指标			动态指标			
	CPU	GPU	存储	CPU	GPU	吞吐率	存储
文献 [1] 方法	√	√	√				
文献 [2-3] 方法				√			√
HMM	√	√	√	√	√	√	√

注:√表示考虑了该指标。

文献 [1] 考虑的均为静态指标,如逻辑运算能力、并行计算能力、神经计算能力、存力、算法能力、路由协议和算效等指标,虽涵盖了目前主流的处理 CPU 和 GPU,以及存储和网络因素,但其均为静态值,没有考虑实时变化的网络状态和算力节点的工作状态,容易导致算力资源匹配准确率低。因为当 2 个算力节点的基础性能相近时,由于无法得知算力节点的实时工作状态,难以辨别哪一个节点更适合用户任务,会出现分配到不合适的算力节点的情况。

文献 [2] 提出了一种基于 Floyd 算法的算力感知路由调度策略来解决智能任务调度问题,其在多任务、多路由节点、多边缘服务器的边缘算力网络场景下做了仿真和模型。但在算力度量的工作上,文献 [2] 仅提出了 2 项算力节点的动态性能指标——CPU 转数和存储剩余量,没有考虑静态性能指标,可能会出现“高分低就”等问题,导致算力资源利用率较低。文献 [3] 研究了算力感知网络中效用优化的资源分配问题,但在算力度量工作中只提到了 CPU 利用率和存储剩余量,同样缺乏静态指标的考虑,也可能造成算力资源利用率低等问题。

静态指标和动态指标均反映算力节点的性能,全面分析算力资源是有效度量算力的方式。通过对现有的算力度量技术的研究与总结,可以看出 HMM 在静态设计和指标综合性的表现更优。

2 方案设计

2.1 设计原则

目前算力度量在业界缺乏统一标识,缺少全面性、综合性和可操作性的设计方案,难以开展算力度量工作。

全面性指算力度量需考虑静态指标和动态指标,不仅要衡量算力节点的基础性能,还要考虑算力节点当前的工作状态,确保全面地评估算力资源。综合性指度量指标需涵盖目前主流算力和网络性能指标,综合度量各类计算资源、存储资源和网络资源等。可操作性指算力度量工作需符合实际且理论和实验均具有可行性,是可实施且有效的度量方案。

1)全面性

考虑全面性工作,本文提出先静后动的混合式度量的设计方案。先计算算力节点基础性能分数,将分数合理分段,最后在段内找到合适的算力节点。

算力节点能否满足用户任务的需求,首先要看它的基础性能高低,其次看它当前的工作状态能否满足用户任务的需求。假设当一个用户任务需要一个高性能且 CPU 空闲率为 30% 的算力节点,这时如果要找到匹配其需求的算力节点,需要从基础性能为高性能的算力节点中选择 CPU 空闲率达到、但不远超 30% 的算力节点。

但有 2 种情况是不可取的,第 1 种情况是不能在所有基础性能未知的节点中选择 CPU 空闲率大于 30% 的算力节点,这样可能会出现当动态指标相同时,本是低性能节点就已满足用户需求,却选择了高性能节点,造成资源浪费和算力节点利用率低的问题;或者动态指标相同时,应选择高性能节点却选择了低性能节点,影响用户利益。第 2 种情况是不能在高性能算力节点中随意选择工作状态无法确定的算力节点,这样容易造成节点匹配准确率低的问题。

区分算力节点基础性能高低,并考虑当前节点工作状态满足业务需求的设计,是一种更有效的度量方式,是用户意图感知和调度算力资源的基础。

2)综合性

指标的综合性在算力度量工作中同样重要。在静态指标选取上,本研究采用 CPU、GPU、存储等因素。在动态指标上,采用 CPU 空闲率、GPU 空闲率、吞吐率和硬盘剩余存储量。综上,本文考虑了基础算力、智能算力、存储能力、网络能力四大指标,并衍生出

9个具体指标.

① 基础算力.当处理普通业务时,CPU可以确定其运行速度^[11].本文将处理器运算能力的性能指标 TOPS/W 和衡量 CPU 速度指标——每秒执行百万级指令数 MIPS 作为静态指标的其中 2 个因素研究.将 CPU 空闲率作为动态指标的其中 1 要素.

② 智能算力.当用于深度学习应用程序、大规模并行处理或其他要求严苛的工作负载时,GPU 就显得尤为重要^[12].本文采用常规神经网络算力指标 GOPS 和存储性能指标 RAM 作为静态指标中的因素,将 GPU 空闲率作为动态指标的其中 1 要素.

③ 存储能力.考虑到算力节点的工作不止有计算,还有存储等功能^[13].本文将存储指标 RAM 和硬盘存储大小考虑为静态指标的因素.将硬盘剩余存储量作为动态指标的其中 1 要素.

④ 网络能力.本文将吞吐率作为动态指标的其

中 1 要素,其单位为 req/s.吞吐率,特指 Web 服务器单位时间内处理的请求数,是服务器并发处理能力的量化描述,是衡量网络服务能力的重要指标^[14].在衡量网络转发性能时,它能直观地反映网络处理事务能力的好坏.

3)可操作性的

为验证本方案的可操作性,图 1 展示了 3 种场景下的案例分析,即对静态指标或动态指标及先静后动的混合式度量方法做了假设和理论分析对比.这 3 种度量方式均存在所有条件都满足的情况.但不同的是,当只考虑静态指标时,其动态指标不一定都能满足用户任务需求,如 CPU 空闲率不足或 GPU 空闲率不足等;当只考虑动态指标时,又存在基础性能不达标或远超标的问题.而 HMM 可以在满足基础性能的同时又能适配算力节点目前的工作状态,可以有效地提高算力资源利用率和匹配准确率.

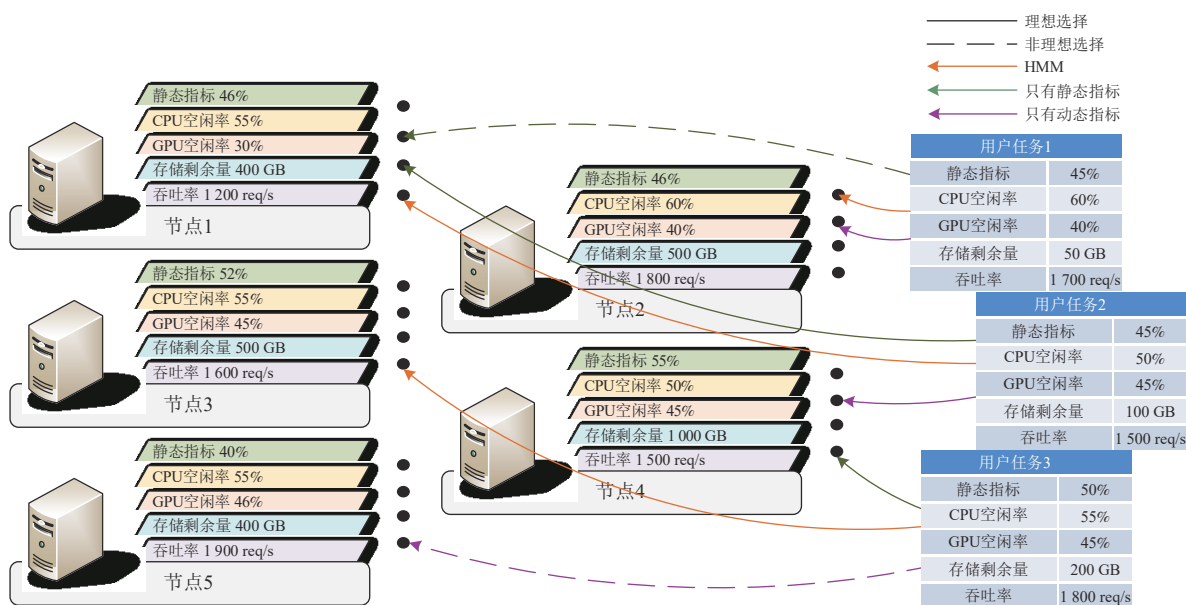


Fig. 1 Case study of HMM, static only and dynamic only indicators

图 1 HMM 与只有静态和只有动态指标的场景案例分析

如图 1 中所示,从用户任务端向左看,每个用户任务都分别引出 3 根不同颜色的线,橘色代表 HMM,绿色代表只有静态指标的方法,紫色代表只有动态指标的方法.而实线和虚线分别代表匹配效果理想和不理想.对于只考虑静态指标的场景,可以看到,用户任务 2、用户任务 3 匹配到了理想的算力节点.但对用户任务 1 来说,其基础性能的需求为要达到待选算力节点群中基础性能的 45%,CPU 空闲率达到 60%,GPU 空闲率达到 40%,存储空间需要 50 GB,吞吐率要达到 1 700 req/s.在选择算力节点时,其匹配到了基

础性能为 46% 的节点 1,但其 CPU 空闲率、GPU 空闲率和吞吐率均不满足要求.

对于只考虑动态指标的场景,可以看到,用户任务 1、用户任务 2 匹配到了理想的算力节点,但对用户任务 3 来说,虽然其匹配到的节点 5 其动态指标均已满足需求,但其 40% 的基础性能不满足需求,也是一个失败的案例.

对于先静后动的混合式度量方法,用户任务 1、用户任务 2 和用户任务 3 分别匹配到了每一项条件都符合的算力节点 2、节点 1 和节点 3,可以看到,HMM 可以精确地匹配到理想的算力节点.

2.2 工作流程

HMM 的工作流程如图 2 所示, 共分为 4 步. 首先利用熵权法计算出算力节点的基础性能分数, 然后利用基于决策树的 CART 算法对分数进行合理的分

段, 其次在符合业务需求的段内, 利用 n 维欧式距离算出算力节点性能与用户任务所需性能的距离, 最后找到最短距离, 即找到最符合用户任务需求的算力节点. 下面介绍具体的算法.

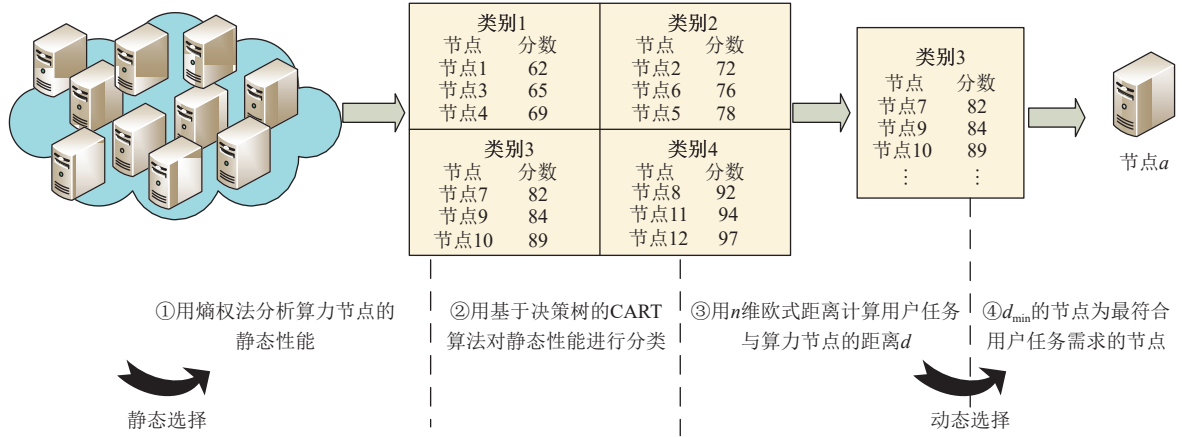


Fig. 2 The workflow of HMM

图2 HMM 的工作流程

2.2.1 计算基础性能得分

首先利用熵权法^[4]对静态指标进行分析, 得到计算节点静态性能的得分. 采用熵权法的目的是由于熵权法是一种客观赋权方法, 其与指标本身无关, 根本在于指标选择是否具有逻辑性、科学性. 它借鉴了信息熵思想, 通过计算指标的信息熵, 根据指标的相对变化程度对系统整体的影响来决定指标的权重, 即根据各个指标标志值的差异程度来进行赋权, 从而得出各个指标相应的权重以及综合评价分数.

假设有 m 个算力节点和 n 个评估指标, 形成原始指标数据矩阵:

$$\begin{pmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{pmatrix}, \quad (1)$$

其中 x_{ij} 表示第 i 个算力节点第 j 项评价指标的数值. 对于某项指标 X_j , 样本的离散程度越大, 则该指标在综合评价中所起的作用就越大; 如果该指标的数值全部相等, 则表示该指标在综合评价中不起作用.

由于本研究中所涉及到的数值是越大越好, 为正向指标, 可不用对各指标进行标准化处理.

计算第 j 个指标中, 第 i 个算力节点的权重:

$$p_{ij} = \frac{x_{ij}}{\sum_{i=1}^n x_{ij}}, \quad (2)$$

其中, $(0 \leq p_{ij} \leq 1, i = 1, 2, \dots, n, j = 1, 2, \dots, m)$. 建立数

据的比重矩阵:

$$\begin{pmatrix} p_{11} & \cdots & p_{1n} \\ \vdots & \ddots & \vdots \\ p_{m1} & \cdots & p_{mn} \end{pmatrix}. \quad (3)$$

然后计算第 j 个指标的熵值:

$$e_j = -k \sum_{i=1}^n p_{ij} \ln p_{ij}, \quad (4)$$

其中, 常数 $k > 0, k = \frac{1}{\ln m}$.

又因为要保证 $0 \leq e_j \leq 1$, 即最大为 1. 所以, 第 j 个指标的熵值为:

$$e_j = -\frac{1}{\ln m} \sum_{i=1}^n p_{ij} \ln p_{ij}, \quad (5)$$

其中, $(i = 1, 2, \dots, n, j = 1, 2, \dots, m)$, 然后需要定义第 j 个指标的差异程度, 熵权法根据各个指标数值的差异程度来进行赋权, 从而得出各个指标相应的权重:

$$d_j = 1 - e_j, \quad (6)$$

其中, $(j = 1, 2, \dots, m)$, 定义权重 w_j :

$$w_j = \frac{d_j}{\sum_{j=1}^m d_j}. \quad (7)$$

最后得到综合评价 F_i :

$$F_i = \sum_{j=1}^m p_{ij} w_j. \quad (8)$$

将 TOPS/W、GOPS、MIPS、RAM、硬盘存储大小

这5个静态因素定义为一个五元组 I , 则 $I = (I_1, I_2, I_3, I_4, I_5)$. 通过熵权法, 取得较为客观的指标权重, 用 w 表示权重, 则 $w_i = \{w_1, w_2, w_3, w_4, w_5\}$ 分别对应5个因素的权重. 从而得到一个多因素静态联合指标 M :

$$M = w_1 \times I_1 + w_2 \times I_2 + w_3 \times I_3 + w_4 \times I_4 + w_5 \times I_5. \quad (9)$$

当用户只对单个算力资源(如存储)有需求时, 则直接对收集到的算力节点的单个资源进行排序. 高性能者享有更高的优先级. 如果是仅对存储有要求, 此时的指标应变成 RAM 和硬盘存储剩余量. 由于硬盘存储剩余量是动态变化的参数, 且它的数值和数值之间在时间上无相关性, 可直接将 RAM 进行优劣排序, 再在此基础上对硬盘存储剩余量进行优劣排序.

2.2.2 对基础性能进行分段

分段可以更为精确地提供算力节点的可选择范围, 是算力度量中的关键性工作.

在第1步得到算力节点基础性能得分后, 第2步利用基于决策树的 CART 算法^[5]来给排好序的算力节点的分数进行分段. CART 可以根据数值本身找到合适的阈值进行分割, 从而达到分段效果.

假设有 K 个类, 算力节点属于第 k 类的概率为 p_k , $k = 1, 2, \dots, K$, 则概率分布的基尼指数定义为:

$$Gini(D) = \sum_{k=1}^K p_k(1-p_k) = 1 - \sum_{k=1}^K p_k^2. \quad (10)$$

给定算力节点的序号为集合 D , 其基尼指数为:

$$Gini = 1 - \sum_{k=1}^K \left(\frac{|C_k|}{|D|} \right)^2, \quad (11)$$

其中, C_k 是 D 中属于第 k 类的样本子集, K 是类的个数.

算力节点的序号集合 D 根据特征值 A 是否取某一可能值 a 被分割成 D_1 和 D_2 2 部分, 特征值 A 为算力节点的性能得分.

$$\begin{aligned} D_1 &= \{(x, y) \in D \mid A(x) = a\}, \\ (D_2 &= D - D_1) \end{aligned} \quad (12)$$

则在特征 A 的条件下, 集合 D 的基尼指数为:

$$Gini(D, A) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2). \quad (13)$$

根据基尼指数, 选择最优特征和最优切分点, 然后得到分类 D_i , 并得到 D_i 中算力节点的数量 C_i .

2.2.3 选择合适的算力节点

考虑到动态因素数值不断变化的特殊性, 在选择评价模型上, 若将用户任务比作 A , 算力节点比作 B , 需求量化为数值时, 需求种类越多, 维数越多. 但 A 和 B 之间的距离越小, 代表差异越小, 即 B 越满足 A 的条件. 基于此思想, 本研究采用 n 维欧氏距离法^[6]对动态因素进行联合分析来选择合适的用户任务的算力节点.

力节点.

假设用户任务需求为 s_i , 实际算力节点的性能为 t_i , 则两者的距离为:

$$d(S, T) = \sqrt{\sum_{i=1}^n (s_i - t_i)^2}. \quad (14)$$

本文将 CPU 空闲率、GPU 空闲率、吞吐率和硬盘剩余量作为动态因素考虑, 式 (14) 中的 $n=4$. $d(S, T)$ 越小, 说明两者的差异越小, 越符合用户任务的需求. 由于这4个因素的量纲不同, 需要做归一化处理, 使结果落到 $[0, 1]$ 区间, 转换函数为:

$$t^* = \frac{t - \min}{\max - \min}. \quad (15)$$

归一化后, 此时的距离为:

$$d(S^*, T^*) = \sqrt{\sum_{i=1}^n (s_i^* - t_i^*)^2}. \quad (16)$$

得到段内的每个距离值后, 最小的距离 $d_{\min}$ 意味着算力性能和用户任务需求更接近, 代表此算力节点是更符合用户任务需求的算力节点.

3 实验评估

3.1 功能评估

根据 2.2 节中的工作流程, 本节实验将分3步进行, 分别对算力节点的基础性能、性能分段和节点选择做出分析. 本研究收集整理了实验室及学校内的 1000 台服务器的算力性能信息 TOPS/W、GOPS、MIPS、RAM、硬盘存储大小, 将其组成了一个数据集.

图3为利用熵权法得到的1000个算力节点基础性能分数的结果. 为了便于展示, 已利用基于决策树的 CART 算法将所得分数分为3类. 图3中可以直观地看到其分数情况呈高中低分布. 算力节点的等级越高, 算力节点的静态性能分数越高.

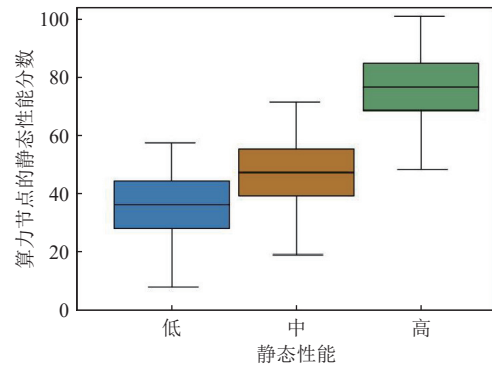


Fig. 3 Comparison of static performance scores of computing nodes

图3 算力节点的静态性能分数对比

在得到的 1000 个算力节点的静态性能分数的基础上,利用基于决策树的 CART 算法得到每一类算力节点的数量,如图 4 所示.此步的目的是为了合理划定基础性能的范围,以便更精准地匹配算力节点.

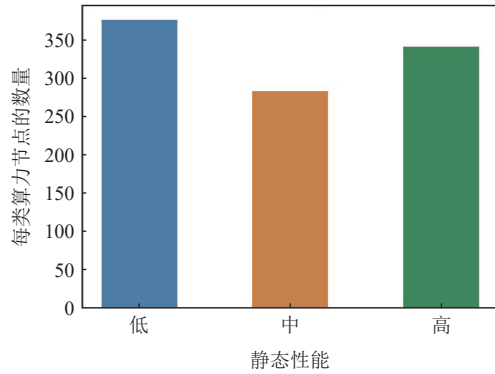


Fig. 4 The number of each type of computing nodes

图 4 每类算力节点的数量

最后根据用户任务的需求确定算力节点基础性能等级,在该等级内利用 n 维欧氏距离,结合 CPU 空闲率、GPU 空闲率、吞吐率和存储剩余量这 4 个动

态因素,判断算力节点的性能与用户任务所需性能的距离,距离最短的节点更符合用户任务的需求.

假设用户任务的需求为:静态性能达到 45%,CPU 空闲率为 60%,GPU 空闲率为 40%,吞吐率为 1700 req/s,存储剩余量需求为 50 GB.为了保证展示效果,在静态性能为 45%~46% 中找到 10 组算力节点的数据进行演示,得到如图 5 所示的结果.柱体表示算力节点性能与用户任务需求之间的距离.由于 4 个动态指标每一个都需要满足用户任务的需求,本研究去掉了距离为“负”的(没达到用户任务需求)的算力节点.在留下的算力节点中比较距离大小,最小的距离为最符合用户任务需求的算力节点.如图 6 所示,节点 3 为匹配该用户任务的最佳算力节点.

3.2 性能评估

本节对提出的 3 种情况,即静态指标、动态指标和 HMM 进行算力节点利用率和匹配准确率的评估对比.

式(17)中算力节点利用率 U 定义为所选算力节点数量与算力节点总量之比,结果如图 7 所示.实验

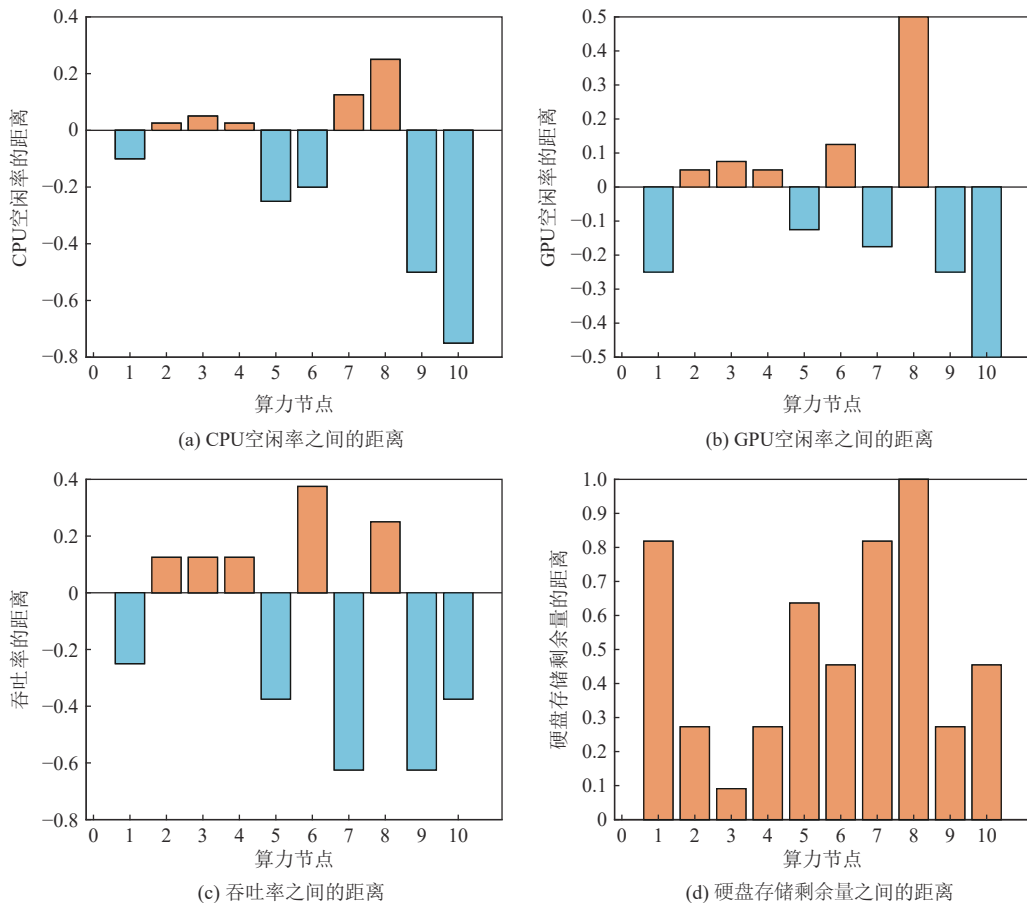


Fig. 5 The distance between the user's task requirements and the performance of the computing nodes

图 5 用户任务需求与算力节点性能的距离

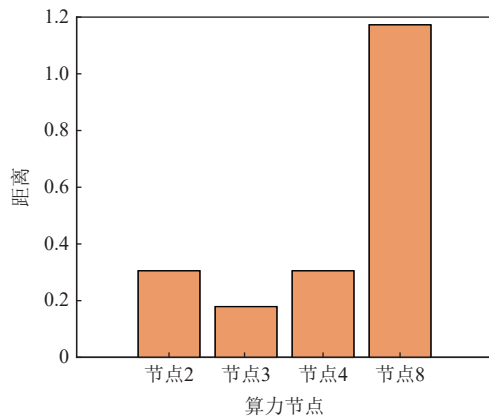


Fig. 6 The final selected computing nodes
图6 最终选出的算力节点

采用了3.1节中的实验,在这个模拟实验中设置了1000个算力节点的静动态算力信息,然后更改用户任务的数量(100, 200, ..., 1000)和需求。可以看到,3种方法的利用率随着节点数的增加均呈上升趋势,采用HMM后,算力节点的利用率明显高于其他2种情况。这是由于既考虑静态指标又考虑动态指标,使得算力资源衡量得愈加精准,用户任务和算力节点能更有效地匹配,进而提升了节点利用率。

$$U = \frac{\text{所选算力节点数量}}{\text{算力节点总量}} \quad (17)$$

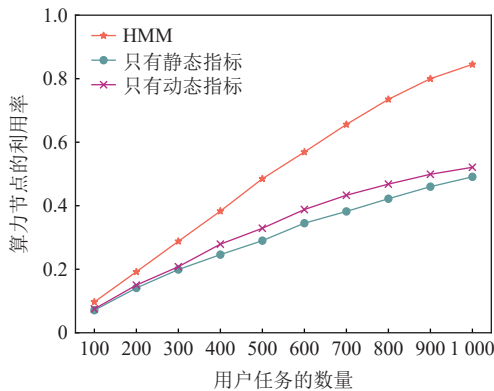


Fig. 7 The evaluation of HMM, static and dynamic metrics for computing nodes utilization
图7 HMM与静态和动态指标算力节点利用率的评估

图8展示了3种方案下算力节点匹配准确率的对比。式(18)中匹配准确率 P^* 定义为符合用户任务所有条件的算力节点数量与所选算力节点数量之比。可以观察到,采用HMM后,由于考虑了度量的全面性、综合性和可操作性,以及实验最后一步 n 维欧式距离去负值找最短的方法,算力节点匹配准确率达到100%,即实现利用及精确的程度。但只考虑静态指标得到的匹配准确率达到30%~40%,只考虑动态指标得到的匹配准确率为35%~50%,可见这2种情

况均不理想。相比之下,HMM的匹配度更优。

$$P^* = \frac{\text{符合用户任务所有条件的算力节点数量}}{\text{所选算力节点数量}} \quad (18)$$

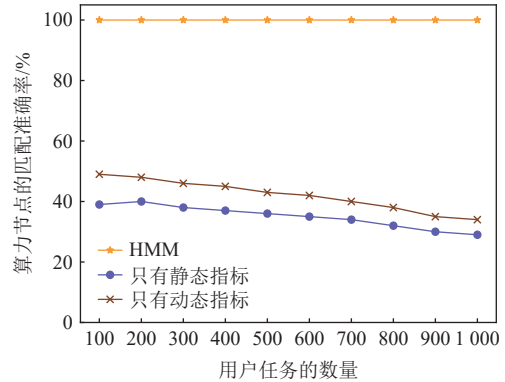


Fig. 8 The evaluation of the matching precision of computing nodes for HMM, static and dynamic indicators
图8 HMM与静态和动态指标算力节点匹配准确率的评估

4 总结

本文设计了先静后动的混合式度量方法HMM。该方法既考虑算力节点的基础性能,又考虑算力节点当前的工作状态,并涵盖了目前主流的算力和网络性能指标,其理论和实验均具有可行性,满足了算力度量的全面性、综合性及可操作性。通过实验数据分析,HMM在提高算力资源利用率和匹配准确率中有良好的表现。后续工作将围绕算力与网络融合开展。未来也将在用户意图感知和算力调度的路由和寻址工作进行更深入的研究。

作者贡献声明: 柴若楠负责实验设计、数据分析,以及论文初稿的撰写; 郜帅指导实验设计、数据分析、论文写作,以及论文修改; 兰江雨参与实验设计和试验结果分析; 刘宁春参与论文的构思及整理。

参考文献

- [1] Qiao Chu. Analysis of computing metric and computing resource scheduling idea[J]. Communication Technology, 2022, 55(9): 1165-1170 (in Chinese)
(乔楚. 算力度量与算网资源调度思路分析[J]. 通信技术, 2022, 55(9): 1165-1170)
- [2] Sun Yukun, Zhang Xing, Lei Bo. Research on intelligent arithmetic-aware routing allocation strategy in edge arithmetic networks[J]. Radio Communication Technology, 2022, 48(1): 60-67 (in Chinese)

- (孙钰坤, 张兴, 雷波. 边缘算力网络中智能算力感知路由分配策略研究[J]. *无线电通信技术*, 2022, 48(1): 60–67)
- [3] Zhao Yuhua, Chong Zheng, Han Xueying, et al. Simulation study of routing mechanism in the computing-aware network[C] //Proc of the 10th Int Conf on Networks, Communication and Computing. Online, 2021: 126–134
- [4] Cheng Qiyue. Structural entropy weighting of evaluation indicators[J]. *Systems Engineering Theory and Practice*, 2010, 30(7): 1225–1228 (in Chinese)
(程启月. 评测指标权重确定的结构熵权法[J]. *系统工程理论与实践*, 2010, 30(7): 1225–1228)
- [5] Quinlan J R. C4. 5: Programs for Machine Learning[M]. England: Elsevier in London, 2014
- [6] Carbó-Dorca R, Besalú E. Geometry of n -dimensional Euclidean space Gaussian enfoldments[J]. *Journal of Mathematical Chemistry*, 2011, 49(10): 2244–2249
- [7] Emma P G. Understanding some simple processor-performance limits[J]. *IBM Journal of Research and Development*, 1997, 41(3): 215–232
- [8] Shekofteh S K, Noori H, Naghibzadeh M, et al. Metric selection for GPU kernel classification[J]. *ACM Transactions on Architecture and Code Optimization*, 2019, 15(4): 1–27
- [9] Wang Haifeng, Chen Qingkui. Energy consumption control model of GPU cluster with multi-index self-optimization[J]. *Journal of Computer Research and Development*, 2015, 52(1): 105–115 (in Chinese)
(王海峰, 陈庆奎. 多指标自趋优的GPU集群能耗控制模型[J]. *计算机研究与发展*, 2015, 52(1): 105–115)
- [10] Zou Qiang. An analytical performance and power model based on the transition probability for hard disks[C] //Proc of the 3rd Int Conf on Awareness Science and Technology (iCAST). Piscataway, NJ: IEEE, 2011: 111–116
- [11] Mastelic T, Brandic I, Jaarevic J. CPU performance coefficient (CPU-PC): A novel performance metric based on real-time CPU resource provisioning in time-shared cloud environments[C] // Proc of the 2014 IEEE 6th Int Conf on Cloud Computing Technology and Science. Piscataway, NJ: IEEE, 2014: 408–415
- [12] Owens J D, Houston M, Luebke D, et al. GPU computing[J]. *Proceedings of the IEEE*, 2008, 96(5): 879–899
- [13] Binh N N, Imai M, Takeuchi Y. A performance maximization algorithm to design ASIPs under the constraint of chip area including RAM and ROM sizes[C] //Proc of 1998 Asia and South Pacific Design Automation Conf. Piscataway, NJ: IEEE, 1998: 367–372
- [14] Liu Boxi. Research on wireless network throughput optimization strategy under limited cache and computing resources[D]. Wu Han: Huazhong University of Science and Technology, 2020 (in Chinese)
(刘铂熙. 缓存、计算资源受限下无线网络吞吐量优化策略研究[D]. 武汉: 华中科技大学, 2020)



Chai Ruonan, born in 1999. PhD candidate. Her main research interests include computer-first network, architecture of ICN, and identifier network.

柴若楠, 1999年生. 博士研究生. 主要研究方向是算力网络、ICN架构、标识网络.



Gao Shuai, born in 1980. PhD, professor. His main research interests include future Internet technology, network security technology.

高帅, 1980年生. 博士, 教授. 主要研究方向是未来互联网技术、网络安全技术.



Lan Jiangyu, born in 1996. PhD candidate. His main research interests include UAV network and edge computing.

兰江雨, 1996年生. 博士研究生. 主要研究方向是无人机网络 and 边缘计算.



Liu Ningchun, born in 1994. PhD candidate. His main research interests include ICN, SDN, and applied cryptography.

刘宁春, 1994年生. 博士研究生. 主要研究方向是 ICN、SDN 和应用密码学.